


```
class LineItem < ActiveRecord::Base
  belongs_to :order
  # . . .
end
```

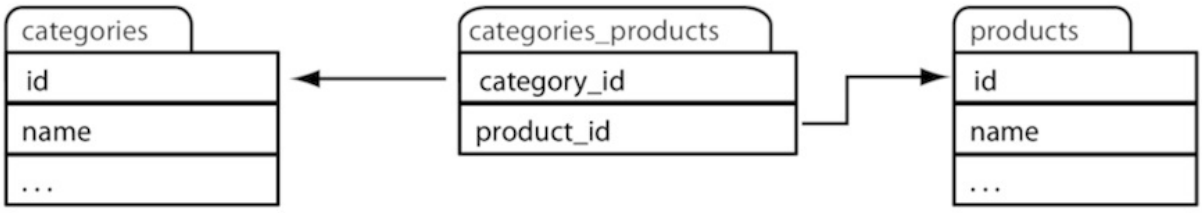
```
class Order < ActiveRecord::Base
  has_many :line_items
  # . . .
end
```

In Active Record, the parent object (the one that logically contains a collection of child objects) uses `has_many` to declare its relationship to the child table, and the child table uses `belongs_to` to indicate its parent. In our example, class `LineItem belongs_to :order`, and the `orders` table `has_many :line_items`.

Note that, again, because the line item contains the foreign key, it has the `belongs_to` declaration.

Many-to-Many Relationships

Finally, we might categorize our products. A product can belong to many categories, and each category may contain multiple products. This is an example of a *many-to-many* relationship. It's as if each side of the relationship contains a collection of items on the other side.




```
class Category< ActiveRecord::Base
  has_and_belongs_to_many :products
  # . . .
end
```

```
class Product< ActiveRecord::Base
  has_and_belongs_to_many :categories
  # . . .
end
```

In Rails we can express this by adding the `has_and_belongs_to_many` declaration to both models.

Many-to-many associations are symmetrical—both of the joined tables declare